

# Azure Service Bus — Grupo de Estudos: **Tópicos, Assinaturas e Mensageria Avançada** (≈1h)

---

 **Público-alvo:** programadores. **Objetivo:** entender **do zero ao avançado** filas, **tópicos & assinaturas, filtros & ações, sessões (FIFO), locks & settlement, DLQ, detecção de duplicidade, AMQP 1.0 & SDKs, limitações, casos de uso, emulador**, e padrões correlatos (Inbox/Outbox, idempotência, etc.).

## Sumário

-  Azure Service Bus — Grupo de Estudos: **Tópicos, Assinaturas e Mensageria Avançada** (≈1h)
  -  Sumário
  -  1. Visão geral & modelos
  -  2. Ciclo de vida das mensagens (end-to-end)
  -  3. Tópicos, Assinaturas, Filtros & Ações
  -  4. Sessões (FIFO, request-response)
  -  5. Transferência, locks e settlement
  -  6. Dead-letter queue (DLQ)
  -  7. Detecção de duplicidade
  -  8. Protocolos, SDKs e depreciações
  -  9. Emulador local
  -  10. Limitações, boas práticas e armadilhas
  -  11. Casos de uso
  -  12. Hands-on (roteiro 60 min)
  -  13. Cheatsheet (TypeScript)
    - 13.1 Enviar para **tópico** (com `messageId` para duplicidade)
    - 13.2 Criar **regra/filtro** da assinatura (Correlation/SQL) — *admin plane*
    - 13.3 Consumir de **assinatura** com `peekLock` (assentando corretamente)
    - 13.4 **Sessions** (receber FIFO por `sessionId` específico)
    - 13.5 **Duplicate detection** (infra + efeito prático)
    - 13.6 **DLQ** (inspecionar e “reatender”)
  -  14. Padrões relacionados (Inbox/Outbox, Idempotência, Sagas)
  -  Referências oficiais

---

## 1. Visão geral & modelos

**Azure Service Bus** é um **message broker gerenciado** para comunicação assíncrona entre serviços, com **filas** (ponto-a-ponto) e **tópicos+assinaturas** (publish/subscribe). Em filas, um consumidor processa cada mensagem; em tópicos, **cada assinatura recebe sua própria cópia** da mensagem.

Fontes: visão geral de entidades ([Queues/Topics/Subscriptions](#)).

### Ideias-chave

- **Tópico** recebe mensagens; **assinaturas** são **filas virtuais** ligadas ao tópico. Consumidores leem **da assinatura** — **não** do tópico diretamente.

- **Filtros & ações** determinam quais mensagens entram em cada assinatura (SQL/boolean/correlation).  
Fontes: [Queues/Topics/Subscriptions](#), [Topic filters & actions](#).

---

## 2. Ciclo de vida das mensagens (end-to-end)

1. **Envio** → produtor publica no **tópico**; o broker **copia** para cada **assinatura** (após filtros).
2. **Armazenamento por assinatura** → a mensagem fica **na assinatura** até ser lida/assentada (settlement), expirar (TTL) ou ir para DLQ.
3. **Recebimento** → consumidor usa **receiveAndDelete** (remove de imediato) ou **peekLock** (lê com lock; decide depois).
4. **Settlement** → **complete** remove **da assinatura**; **abandon** libera para redelivery; **deadLetter** envia ao DLQ; **defer** posterga (recupera por **sequenceNumber**).
5. **Finalização** → **assinaturas criadas depois não** recebem mensagens antigas (sem replay retroativo).  
Fontes: [Transfers](#), [locks](#), [settlement](#), quickstarts ([.NET](#), [JS](#)).

💡 **Importante:** a remoção efetiva ocorre **por assinatura** (fila virtual). O tópico publica/roteia; o “fim de vida” acontece na **entidade de assinatura** durante o settlement.

---

## 3. Tópicos, Assinaturas, Filtros & Ações

Cada assinatura possui **regras**; cada regra = **filtro** + (opcional) **ação**.

- Regras **sem ação** são combinadas com **OR** e geram **uma única cópia**.
- **Tipos de filtro:** **SQL**, **Boolean** ([TrueFilter](#) / [FalseFilter](#)) e **Correlation**.
- Exemplos (portal/CLI/SDK): [Filtros & Ações](#) e [Exemplos de filtros](#).
- **Sintaxe:** [SQL Filter](#) e [SQL Rule Action](#).

---

## 4. Sessões (FIFO, request-response)

**Sessions** permitem processar **sequências ordenadas** (FIFO) por **sessionId** e manter **estado da sessão** (Standard/Premium). Útil para **request-response** correlacionado. **Basic não suporta sessions**.

Fonte: [Message sessions](#).

---

## 5. Transferência, locks e settlement

- **receiveAndDelete**: remove ao entregar (rápido, sem reprocesso).
- **peekLock**: entrega com **lock**; a aplicação chama **complete/abandon/deadLetter/defer**.
- **Renovação de lock** é necessária para processamentos longos.

Fontes: [Transfers](#), [locks](#), [settlement](#), API TS: [ServiceBusReceiverOptions](#).

---

## 6. Dead-letter queue (DLQ)

Cada **fila** e **assinatura** tem uma **DLQ** associada (subfila implícita). Armazena mensagens **improcessáveis** (ex.: **MaxDeliveryCount** excedido, TTL expirado, erro de processamento/roteamento). A **limpeza é sua** (leitor do

DLQ).

Fonte: [DLQ](#).

---

## 7. Detecção de duplicidade

Evita reprocessar mensagens **idênticas** (por `messageId`) dentro de uma **janela configurável**; o broker **descarta** duplicatas (envio duplicado retorna sucesso).

Fontes: [Duplicate detection](#), [Enable duplicate detection \(CLI\)](#).

---

## 8. Protocolos, SDKs e deprecacões

- Protocolo principal: **AMQP 1.0** (padrão aberto, binário, confiável).
- **SDKs antigos** e **SBMP** serão descontinuados **em 30/09/2026** → migre para [@azure/service-bus](#).  
Fontes: [AMQP guide \(SB\)](#), [Transfers/Locks/Settlement](#).

### Leituras AMQP (extra):

- OASIS AMQP 1.0 (PDF): <https://docs.oasis-open.org/amqp/core/v1.0/os/amqp-core-complete-v1.0-os.pdf>
  - AMQP overview (SB): <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-amqp-overview>
  - AMQP request/response (SB): <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-amqp-request-response>
  - AMQP.org downloads: <https://www.amqp.org/resources/download>
- 

## 9. Emulador local

- Overview: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/test-locally-with-service-bus-emulator>
  - Docker Hub: <https://hub.docker.com/r/microsoft/azure-messaging-servicebus-emulator>
- 

## 10. Limitações, boas práticas e armadilhas

- **Assinaturas são independentes**: cada uma tem sua **cópia**; settlement (`complete/abandon/deadLetter/defer`) afeta **apenas aquela assinatura**.
  - **Sessions** só em **Standard/Premium**; planeje **partition key** e `sessionId` quando usar partições + sessão.
  - **Filtros SQL complexos** em muitas assinaturas podem impactar throughput; prefira **CorrelationFilter** quando possível.
  - **Observabilidade**: monitore **DLQ**, **MaxDeliveryCount**, **latência**, **lock renewals**; implemente **correlação**, **retries** e **idempotência**.
- 

## 11. Casos de uso

- **Fan-out de eventos** (pub/sub) com assinaturas filtradas (faturamento, analytics, notificações).

- **Fluxos FIFO por entidade** (ex.: pedidos/contas) com **sessions**.
- **Orquestração resiliente** com **peekLock** + **DLQ** + **duplicate detection**.

---

## 🔧 12. Hands-on (roteiro 60 min)

1. **(10 min)** Criar **tópico** e **duas assinaturas** (ex.: **billing**, **analytics**).
2. **(10 min)** Configurar **filtros** (SQL + correlation) para separar tipos de eventos.
3. **(10 min)** Publicar eventos; consumir com **peekLock**; testar **abandon** e redelivery.
4. **(10 min)** Habilitar **sessions** e demonstrar **FIFO** por **sessionId**.
5. **(10 min)** Gerar mensagem problemática e inspecionar a **DLQ**.
6. **(10 min)** Ligar **duplicate detection** e validar com **messageId** repetido.

---

## 📄 13. Cheatsheet (TypeScript)

Pré-requisitos: `npm i @azure/service-bus` e `SB_CONN` (connection string).

### 13.1 Enviar para **tópico** (com **messageId** para duplicidade)

```
import { ServiceBusClient, ServiceBusMessage } from "@azure/service-bus";

const conn = process.env.SB_CONN!;
const topicName = "orders";

export async function sendOrder(order: { id: string; customerId: string; total: number; color?: string; priority?: "low"|"normal"|"high" }) {
  const sb = new ServiceBusClient(conn);
  const sender = sb.createSender(topicName);

  const msg: ServiceBusMessage = {
    body: order,
    messageId: order.id,
    subject: "order.created",
    applicationProperties: {
      color: order.color ?? "none",
      priority: order.priority ?? "normal",
    },
  };

  await sender.sendMessage(msg);
  await sender.close();
  await sb.close();
}
```

### 13.2 Criar **regra/filtro** da assinatura (Correlation/SQL) — *admin plane*

```
import { ServiceBusClient, CorrelationRuleFilter, SqlRuleFilter } from
"@azure/service-bus";
```

```

const conn = process.env.SB_CONN!;
const topicName = "orders";
const subName = "analytics";

export async function configureFilters() {
  const sb = new ServiceBusClient(conn);
  const admin = sb.createAdministrationClient();

  // Correlation: subject=order.created AND 'high' como marca de correlação
  await admin.createRule(topicName, subName, {
    name: "only-high-priority",
    filter: new CorrelationRuleFilter({ subject: "order.created", correlationId:
"high" })
  }).catch(async e => {
    if ((e as any).code !== "Conflict") throw e;
  });

  // SQL filter sobre applicationProperties
  await admin.createRule(topicName, subName, {
    name: "color-blue",
    filter: new SqlRuleFilter("user.color = 'blue' AND sys.Label IS NULL"),
  }).catch(async e => {
    if ((e as any).code !== "Conflict") throw e;
  });

  await sb.close();
}

```

### Notas

- **CorrelationRuleFilter** casa campos específicos (ex.: **messageId**, **subject**, **sessionId**); para **applicationProperties** arbitrárias, prefira **SqlRuleFilter**.
- Regras **sem ação** são **OR**; múltiplas correspondências **não** duplicam a mensagem na assinatura.

## 13.3 Consumir de **assinatura** com **peekLock** (assentando corretamente)

```

import { ServiceBusClient, ProcessErrorArgs, ServiceBusReceivedMessage } from
"@azure/service-bus";

const conn = process.env.SB_CONN!;
const topicName = "orders";
const subName = "billing";

export async function startBillingProcessor() {
  const sb = new ServiceBusClient(conn);

  const processor = sb.createProcessor(topicName, subName, {
    receiveMode: "peekLock",
    maxConcurrentCalls: 4,
    autoCompleteMessages: false,
  });
}

```

```

    });

    processor.processMessage = async (args) => {
        const msg: ServiceBusReceivedMessage = args.message;
        try {
            const order = msg.body as { id: string; total: number };
            // ... processar cobrança ...
            await args.completeMessage(msg);
        } catch (err) {
            await args.abandonMessage(msg, { propertiesToModify: { retryAt: Date.now() } }
        });
    }
    };

    processor.processError = async (args: ProcessErrorArgs) => {
        console.error("SB error", args.error);
    };

    await processor.start();
}

```

### 13.4 Sessions (receber FIFO por `sessionId` específico)

```

import { ServiceBusClient, ServiceBusReceivedMessage } from "@azure/service-bus";

const conn = process.env.SB_CONN!;
const topicName = "orders";
const subName = "accounting";

export async function receiveForAccount(accountId: string) {
    const sb = new ServiceBusClient(conn);
    const receiver = await sb.acceptSession(topicName, subName, accountId);

    for await (const msg of receiver.getMessageIterator()) {
        const body = msg.body as any;
        // mensagens dessa sessionId chegam em ordem
        await receiver.completeMessage(msg);
    }

    await receiver.close();
    await sb.close();
}

```

### 13.5 Duplicate detection (infra + efeito prático)

```

# Janela de 1 dia para duplicidade
az servicebus topic create \
  --resource-group rg \
  --namespace-name ns \

```

```
--name orders \
--enable-duplicate-detection true \
--duplicate-detection-history-time-window P1D
```

Envios com o **mesmo messageId** dentro da janela são **descartados** pelo broker (sem duplicar).

### 13.6 DLQ (inspecionar e “reatender”)

```
import { ServiceBusClient } from "@azure/service-bus";

export async function reprocessDlq() {
  const sb = new ServiceBusClient(process.env.SB_CONN!);
  const receiver = sb.createReceiver("orders", "billing", { subQueueType:
    "deadLetter" });

  for await (const msg of receiver.getMessageIterator()) {
    try {
      // tente corrigir e reenfileirar em outra entidade
      await receiver.completeMessage(msg);
    } catch {
      await receiver.abandonMessage(msg);
    }
  }

  await receiver.close();
  await sb.close();
}
```

## 14. Padrões relacionados (Inbox/Outbox, Idempotência, Sagas)

 **Vale a leitura** para sistemas distribuídos confiáveis em cima de brokers (inclusive Service Bus).

- **Outbox Pattern (Transactional Outbox):** grava o “evento” na mesma transação do **write** no banco, e um **dispatcher** assíncrono publica no broker (evita “half-commits”).
  - Martin Fowler — Outbox: <https://martinfowler.com/articles/patterns-of-distributed-systems/outbox.html>
  - Debezium — Outbox Event Router: <https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html>
  - NServiceBus — Outbox: <https://docs.particular.net/nservicebus/outbox/>
- **Inbox Pattern:** persistir **mensagens recebidas** e verificar **deduplicação/idempotência** antes de processar (evita reprocesso em redeliveries).
  - Chris Richardson — Reliable Messaging: <https://microservices.io/patterns/communication-style/messaging.html>
  - Microservices.io — Idempotent Consumer: <https://microservices.io/patterns/communication-style/idempotent-consumer.html>

- **Idempotência & Exactly-Once (semântica prática):** em brokers, **alvo é “at-least-once + idempotência”** (exactly-once fim-a-fim é raríssimo e caro).
    - Azure SB — Duplicate detection: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/duplicate-detection>
    - Uber Engineering — Idempotency Keys (inspiração): <https://eng.uber.com/idempotent-rest-apis/>
  - **Sagas (Process Manager/Orchestration):** coordenar transações distribuídas por **mensagens** com **compensação**.
    - Microservices.io — Saga: <https://microservices.io/patterns/data/saga.html>
    - Particular/NServiceBus — Sagas: <https://docs.particular.net/nservicebus/sagas/>
  - **Enterprise Integration Patterns (EIP):** catálogo clássico de padrões de mensageria (roteamento, filtros, DLQ, etc.).
    - Catálogo EIP: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/>
- 

## Referências oficiais

- **Entidades:** Queues, Topics, Subscriptions — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-queues-topics-subscriptions>
- **Filtros & Ações** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/topic-filters>
- **Exemplos de filtros** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-filter-examples>
- **SQL Filter** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-sql-filter>
- **SQL Rule Action** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-sql-rule-action>
- **Sessions** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/message-sessions>
- **Transfers, locks, settlement** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/message-transfers-locks-settlement>
- **Receiver options (TS)** — <https://learn.microsoft.com/en-us/javascript/api/%40azure/service-bus/servicebusreceiveroptions>
- **Duplicate detection** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/duplicate-detection>
- **Enable duplicate detection (CLI)** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/enable-duplicate-detection>
- **AMQP 1.0 (SB guide)** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-amqp-protocol-guide>
- **AMQP overview (SB)** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-amqp-overview>
- **AMQP request/response (SB)** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-amqp-request-response>
- **AMQP 1.0 (OASIS PDF)** — <https://docs.oasis-open.org/amqp/core/v1.0/os/amqp-core-complete-v1.0-os.pdf>
- **Emulador** — <https://learn.microsoft.com/en-us/azure/service-bus-messaging/test-locally-with-service-bus-emulator>

- **Docker do emulador** — <https://hub.docker.com/r/microsoft/azure-messaging-servicebus-emulator>